>_ MCMP113 · Week 2

Flowcharts and Their Symbols

Basic flowchart examples · from pictures to pseudocode to a first look at C#

Asst. Prof. Hasan Oğuz, PhD
Introduction to Basic Algorithms and Programming · Istanbul Okan University
Vocational School · 2026–2027 Fall

How to use these notes

These notes follow the Week 2 slides and add what does not fit on a slide: every flowchart of the lecture drawn in full, the matching pseudocode, trace tables, test cases, and the C# "look-ahead" programs from the `code/` folder together with real sample runs. **You are not expected to understand the C# syntax yet** – it is taught from Week 3 on. Read the programs only to see that every box of a flowchart becomes one or two lines of code. Work through Sections 1–3 before Week 3, draw every example yourself on paper, then do the exercises in Section 7. Difficulty: ⋆ basic, ⋆⋆ intermediate, ⋆⋆⋆ challenging.

Contents

1 Flowchart symbols 2

2 Three structures as flowchart patterns 4

3 Basic flowchart examples 6

4 Common mistakes and self-checks 18

5 Tools for drawing flowcharts 19

6 Summary 20

7 Exercises 20

References and further reading 21

Learning outcomes

After this week you should be able to:

1. name and draw the standard flowchart symbols (ISO 5807) and explain what each one means;
2. apply the drawing rules to produce clear and correct flowcharts;
3. recognise sequence, selection and repetition as flowchart patterns, and nest them;

4. convert pseudocode into a flowchart and a flowchart into pseudocode;
5. trace a flowchart by hand with a trace table and predict its output;
6. relate each box of a flowchart to the corresponding line(s) of a C# program (preview).

>_ 1 Flowchart symbols

1.1 What is a flowchart?

Definition

A **flowchart** is a diagram of an algorithm. Each *shape* is one step, and the shape tells you what *kind* of step it is. *Flow lines* (arrows) show the order in which the steps are executed.

In Week 1 we wrote algorithms in pseudocode. A flowchart describes exactly the same algorithm, but visually: branches (IF) and loops (WHILE) become paths you can follow with your finger. Flowcharts are language-independent – a C#, Python or Java programmer can read the same chart – and they are a standard exam tool, drawn by hand like pseudocode.

A short history. Flowcharts are older than computers. In 1921 Frank and Lillian Gilbreth presented the *flow process chart* to the American Society of Mechanical Engineers as a tool for analysing factory work. In 1947 Herman Goldstine and John von Neumann used *flow diagrams* to plan programs for early electronic computers. The symbols were standardised in the USA by ANSI X3.5 (1970) and internationally by **ISO 5807** (1985), which is why textbooks all over the world use the same shapes. Modern notations such as UML activity diagrams and BPMN, and teaching tools such as Flowgorithm, are built on the same idea.

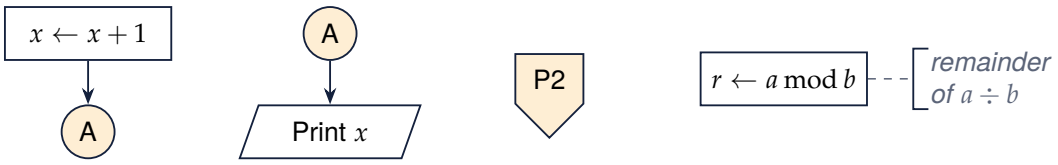
1.2 The six basic symbols

Symbol	Name	Meaning and rules
Start	**Terminal**	Start or end of the algorithm. Exactly one Start; in this course also one End.
Read x	**Input / Output**	**READ** data from the keyboard or **PRINT** results on the screen.
$sum \leftarrow sum + x$	**Process**	A calculation or an assignment ($\leftarrow$).
$x > 0$?	**Decision**	A yes/no question. Exactly two exits, labelled *yes* and *no* (or *true*/*false*).
GCD(a, b)	**Predefined process**	A named sub-algorithm that is described elsewhere (on another page). In C# it becomes a method call (Week 12).
A	**Connector**	Joins flow lines, or continues the chart at another circle with the same letter.

1.3 Flow lines, connectors and annotations

- **Flow lines** connect the symbols. The main direction is top to bottom and left to right, but *always* draw the arrowhead, because loops flow upwards.

- An **on-page connector** is a small circle with a letter; two circles with the same letter mean "the line continues here". Use connectors to avoid crossing lines or to split a long chart.
- An **off-page connector** (a "home-plate" pentagon) continues the chart on another page.
- An **annotation** is an open bracket joined to a symbol by a dashed line. It is a comment: it is not a step and is never executed, exactly like `//` comments in C#.



on-page connectors off-page connector annotation

1.4 Every box has a pseudocode line

The table below is our dictionary between the three notations. Note that the decision diamond is used both for **IF** and for **WHILE**; what turns it into a loop is an arrow that goes back *up* (Section 2).

Symbol	Pseudocode (Week 1)	C# (from Week 3)
Terminal	**START** … **END**	the program begins / ends
Input	**READ** x	`x = int.Parse(Console.ReadLine());`
Output	**PRINT** x	`Console.WriteLine(x);`
Process	$x \leftarrow x + 1$	`x = x + 1;`
Decision	**IF** $x < 0$ **THEN** … / **WHILE** $i \leq N$ …	`if (x < 0)` … / `while (i <= n)` …
Predefined process	use the sub-algorithm GCD(a, b)	a method call, e.g. `Math.Sqrt(x)` (Week 12)

Assignment is not comparison

In pseudocode and flowcharts, $\leftarrow$ *assigns* and $=$ *compares*. In C#, `=` assigns and `==` compares.

1.5 Eight drawing rules

These rules are also the grading checklist for flowchart questions in the exams.

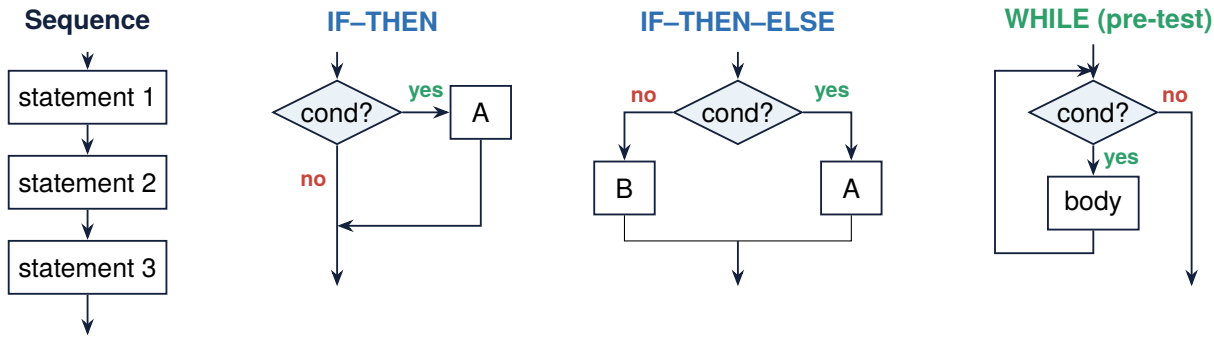
1. **One Start, one End**; every path from Start must reach End.
2. **Arrows on every line**; main direction top $\rightarrow$ bottom and left $\rightarrow$ right.
3. **A decision has exactly two exits**, labelled yes/no (true/false).
4. **Every other box has one way in and one way out**; branches join again before the flow continues.
5. **No crossing lines**: re-route the line or use a connector.
6. **Short, precise text**: "$sum \leftarrow sum + i$", not "add it up".
7. **Loops must make progress**: something that the condition tests must change inside the loop.
8. **One idea per box, same names as the pseudocode**: never mix an input and a calculation ("Read x, then double it" needs two boxes).

Quiz: which symbol?

Which symbol would you use for: (1) Read age; (2) $total \leftarrow total + price$; (3) $age \geq 18$?; (4) Print "Welcome"; (5) GCD(a, b), described on page 2; (6) Start; (7) Read x, then $x \leftarrow 2 \times x$? *Answers:* 1 and 4 input/output; 2 process; 3 decision; 5 predefined process; 6 terminal; 7 is a trap – two boxes, an input followed by a process.

>_ 2 Three structures as flowchart patterns

Week 1 showed that every algorithm is built from three structures: **sequence**, **selection** and **repetition** (Böhm and Jacopini, 1966). Each has a standard flowchart pattern with **exactly one entry arrow and one exit arrow**. Because of this, a whole pattern can be placed wherever a single box could stand: an IF inside a loop, a loop inside an ELSE, and so on. This is called **nesting**.



Sequence	IF–THEN	IF–THEN–ELSE	WHILE
statement 1 statement 2 statement 3	**IF** *c* **THEN** A **END IF**	**IF** *c* **THEN** A **ELSE** B **END IF**	**WHILE** *c* body **END WHILE**

Selection. In the IF–THEN pattern the "no" exit simply skips the block A; in IF–THEN–ELSE exactly one of A or B runs, and then the two paths join. The join is essential: if the "yes" path of an IF went straight to End, everything after the IF would be skipped.

Repetition. The only difference between an IF and a loop is the **back-edge**: an arrow from the end of the body back to a point *above* the decision. **An arrow pointing upwards always means repetition.** In a **pre-test loop** (WHILE) the condition is tested first, so the body may run zero times. In a **post-test loop** the body comes first and the decision after it, so the body runs at least once; C# has the `do … while` statement for this (Week 7). Our pseudocode only uses WHILE and FOR; a counting loop **FOR** $i \leftarrow 1$ **TO** N is drawn as $i \leftarrow 1$, the decision $i \leq N$, the body, $i \leftarrow i + 1$ and the back-edge – exactly the shape of Example 7 (Section 3.7). We draw the back-edge into the line just above the diamond, so that the diamond keeps a single entry.

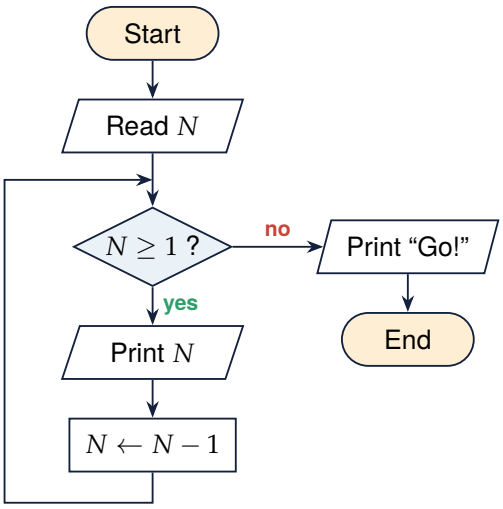
2.1 Conversion 1: pseudocode → flowchart

Convert line by line: **START**/**END** become terminals, **READ** /**PRINT** become parallelograms, assignments become rectangles, a **WHILE** line becomes a diamond, and **END WHILE** becomes

the back-edge. The statement after **END WHILE** hangs on the "no" exit. Example: the countdown of Week 1 (exercise D).

Algorithm 1 – Countdown

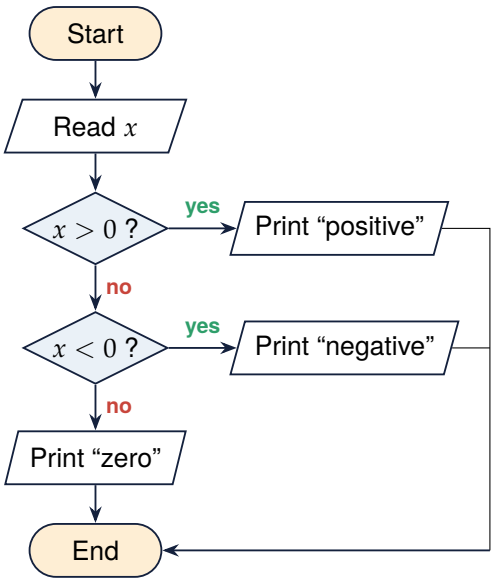
```
1 START
2 READ N
3 WHILE N ≥ 1
4     PRINT N
5     N ← N − 1
6 END WHILE
7 PRINT "Go!"
8 END
```



Trace for $N = 3$: the output is $3, 2, 1$, Go!. Edge case $N = 0$: the condition is false at once and only "Go!" is printed. If the box $N \leftarrow N - 1$ is forgotten, $N \geq 1$ stays true forever: an infinite loop.

2.2 Conversion 2: flowchart → pseudocode

Read the chart from Start. For every diamond ask: *do its two paths join again further down (IF), or does an arrow return above it (WHILE)?* In the chart below there is no upward arrow, so it contains only selection; the second decision sits on the "no" exit of the first, so the second IF goes *inside the ELSE* of the first.



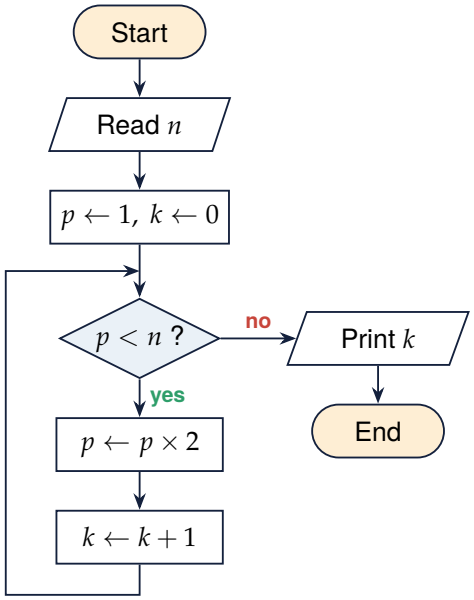
Algorithm 2 – Sign of a number

```
1  START
2  READ x
3  IF x > 0 THEN
4      PRINT "positive"
5  ELSE
6      IF x < 0 THEN
7          PRINT "negative"
8      ELSE
9          PRINT "zero"
10     END IF
11 END IF
12 END
```

Three test inputs visit every box: $5 \rightarrow$ positive (path yes), $-3 \rightarrow$ negative (no, yes), $0 \rightarrow$ zero (no, no). Writing three separate IFs ($x > 0$, $x < 0$, $x = 0$) gives the same output here, but it re-tests conditions that are already known and no longer matches the chart. In Week 5 this shape is written with `else if`.

2.3 Tracing a flowchart with a trace table

To **trace** (desk-check) a flowchart, put a finger on Start and follow the arrows. Every time a box changes a variable, write the new value in a new row of a **trace table**; every time you pass a decision, write down the answer. The chart below is deliberately given without explanation. Trace it for $n = 20$.



step	$p < n$?	p	k	output
start	–	1	0	
1	yes	2	1	
2	yes	4	2	
3	yes	8	3	
4	yes	16	4	
5	yes	32	5	
exit	no	32	5	**5**

What does it compute? p runs through $1, 2, 4, 8, \ldots$, so after k passes $p = 2^k$, and the loop stops at the first k with $2^k \geq n$. The output is the number of doublings needed to reach at least n, i.e. the smallest k with $2^k \geq n$ (for $n \geq 1$ this is $\lceil \log_2 n \rceil$). For $n = 1000$ it prints 10, because $2^9 = 512 < 1000 \leq 1024 = 2^{10}$ – the same 10 as the worst case of the halving guessing game of Week 1. For $n = 1$ the body never runs and it prints 0.

>_ 3 Basic flowchart examples

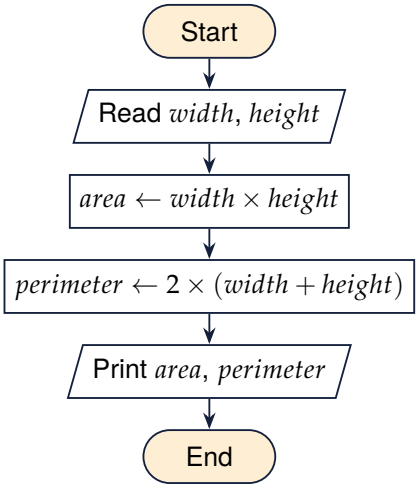
Every example follows the same path: *problem* → *flowchart* → *C# preview* → *a real run*. The programs are in the `code/` folder (one project per example; run one with `cd code/02_07_SumToN` and `dotnet run`). In the sample runs, what the user types is shown in **amber**.

About the C# previews

The programs use top-level statements, `int.Parse`/`double.Parse` for input, `if`/`else` and `while`. All of this is explained properly in Weeks 3–7. For now, look only for the correspondence: each input box is a `Console.ReadLine()`, each process box an assignment, each diamond an `if` or a `while`, each output box a `Console.WriteLine`.

3.1 Example 1 – rectangle area and perimeter (sequence)

Problem. Read the width and height of a rectangle; print its area $A = w \cdot h$ and perimeter $P = 2(w + h)$. Pure sequence: six boxes, every box runs exactly once.



Sample run

```
Width: 4.5
Height: 3

Area: 13.5
Perimeter: 15
```

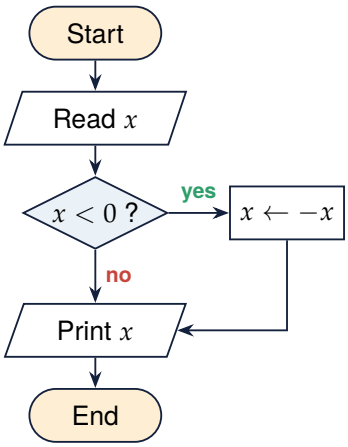
Check by hand: $4.5 \times 3 = 13.5$ and $2 \times (4.5 + 3) = 15$. A negative width would produce a negative area without complaint; rejecting invalid input needs selection (Week 5).

Listing 1: `code/02_01_RectangleArea/Program.cs` – the comments name the flowchart box of each line

```
1  // MCMP113 · Week 2 · Example 1 — Rectangle area and perimeter
2  // Sequence: every box of the flowchart runs exactly once, top to bottom. (Preview: syntax from
       Week 3.)
3
4  Console.Write("Width: ");
5  double width = double.Parse(Console.ReadLine());    // input
6  Console.Write("Height: ");
7  double height = double.Parse(Console.ReadLine());   // input
8
9  double area = width * height;                       // process
10 double perimeter = 2 * (width + height);            // process
11
12 Console.WriteLine($"Area: {area}");                 // output
13 Console.WriteLine($"Perimeter: {perimeter}");       // output
```

3.2 Example 2 – absolute value (IF–THEN)

The Week 1 example. The "no" exit skips the process box; both paths meet at the output box.



Two runs

```
Enter a number: -7.25
Absolute value: 7.25
Enter a number: 12
Absolute value: 12
```

Listing 2: `code/02_02_AbsoluteValue/Program.cs`

```
// MCMP113 · Week 2 · Example 2 — Absolute value
// Selection with one branch (IF ... THEN): the "no" exit simply skips the process box.

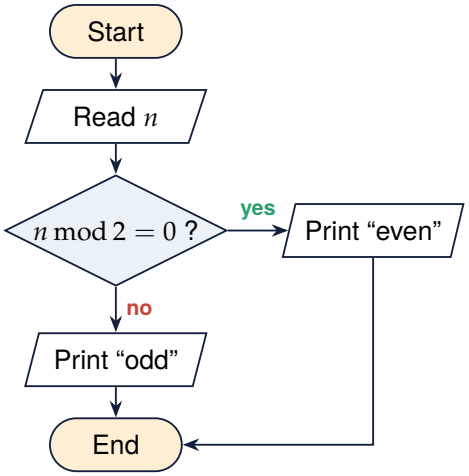
Console.Write("Enter a number: ");
double x = double.Parse(Console.ReadLine());

if (x < 0)
{
    x = -x;          // "yes" branch: remove the minus sign
}

Console.WriteLine($"Absolute value: {x}");
```

3.3 Example 3 – even or odd (IF–THEN–ELSE)

An integer n is even exactly when $n \bmod 2 = 0$ (C#: `n % 2 == 0`). Exactly one of the two output boxes is used.



Two runs

```
Enter an integer: 14
14 is even
Enter an integer: -7
-7 is odd
```

Listing 3: `code/02_03_EvenOdd/Program.cs`

```
// MCMP113 · Week 2 · Example 3 — Even or odd
// Selection with two branches (IF ... THEN ... ELSE): exactly one of the two outputs is printed.

Console.Write("Enter an integer: ");
int n = int.Parse(Console.ReadLine());

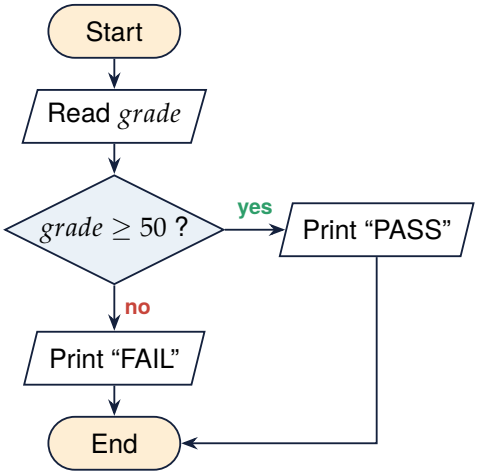
if (n % 2 == 0)      // % is the remainder operator (mod)
{
    Console.WriteLine($"{n} is even");
}
else
{
    Console.WriteLine($"{n} is odd");
}
```

Test for 0, not for 1

In C# the remainder of a negative number is negative or zero: `-7 % 2` is -1. A program that tests “odd” with `n % 2 == 1` would therefore call -7 even. Testing `n % 2 == 0` for “even” works for all integers. Tests: 14, -7 and the edge case 0 (even).

3.4 Example 4 – grade: PASS or FAIL (a boundary)

The specification says that a grade of 50 or more passes. The condition is therefore $grade \geq 50$ (C#: >=), and the most important test is exactly 50.



Two runs

```
Grade (0-100): 49.5
FAIL
Grade (0-100): 50
PASS
```

test	grade	expected
lowest	0	FAIL
just below	49.5	FAIL
boundary	**50**	**PASS**
highest	100	PASS

Listing 4: `code/02_04_PassFail/Program.cs`

```
// MCMP113 · Week 2 · Example 4 — Grade: PASS or FAIL
// Two-way decision with a boundary: exactly 50 must be a pass, so the condition is grade >= 50.

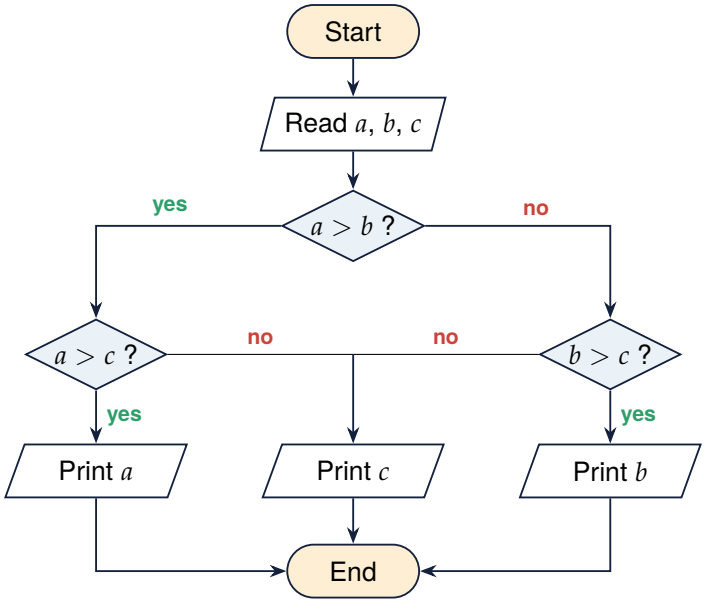
Console.Write("Grade (0-100): ");
double grade = double.Parse(Console.ReadLine());

if (grade >= 50)
{
    Console.WriteLine("PASS");
}
else
{
    Console.WriteLine("FAIL");
}
```

If someone wrote $grade > 50$, only the boundary test would reveal the bug. Invalid grades such as 120 or -5 are not rejected yet (Week 5).

3.5 Example 5 – maximum of three numbers (nested decisions)

Method 1: nested decisions. First compare a and b; the larger one is then compared with c. Two different decisions can end in "Print c", so both of those exits join into a single box.

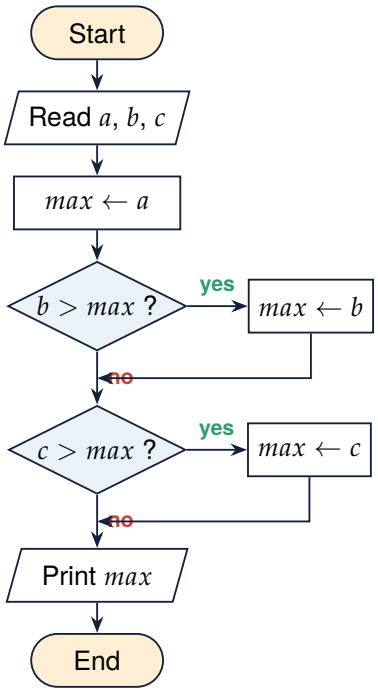


The chart has four paths, so we need at least four tests; negative numbers and ties must work too:

a, b, c	decisions	path ends in	maximum
$4, 9, 7$	$a > b$ no, $b > c$ yes	Print b	9
$8, 3, 8$	$a > b$ yes, $a > c$ no	Print c	8
$2, 5, 9$	$a > b$ no, $b > c$ no	Print c	9
$9, 1, 4$	$a > b$ yes, $a > c$ yes	Print a	9
$-2, -5, -1$	$a > b$ yes, $a > c$ no	Print c	-1

For the tie $(8, 3, 8)$ the chart prints c; that is still correct, because c equals the maximum.

Method 2: a running maximum. Keep "the largest so far" in *max* and let each new number challenge it: two IF–THEN patterns in sequence, no nesting. This design scales – for 100 numbers it is the same two boxes inside a loop (Week 6) – whereas the nested design would explode. Starting with $max \leftarrow a$ (not with 0) is what makes negative inputs work.



The complete program contains both methods (Method 1 as nested `if` statements) and prints both results, so you can see that they agree:

Sample run

```
a = 4
b = 9
c = 7
Method 1 (nested): 9
Method 2 (running max): 9
```

Negative numbers

```
a = -2
b = -5
c = -1
Method 1 (nested): -1
Method 2 (running max): -1
```

Listing 5: `code/02_05_MaxOfThree/Program.cs`

```
// MCMP113 · Week 2 · Example 5 — Maximum of three numbers
// Two designs for the same problem: nested decisions vs. a running "largest so far".

Console.Write("a = ");
int a = int.Parse(Console.ReadLine());
Console.Write("b = ");
int b = int.Parse(Console.ReadLine());
Console.Write("c = ");
int c = int.Parse(Console.ReadLine());

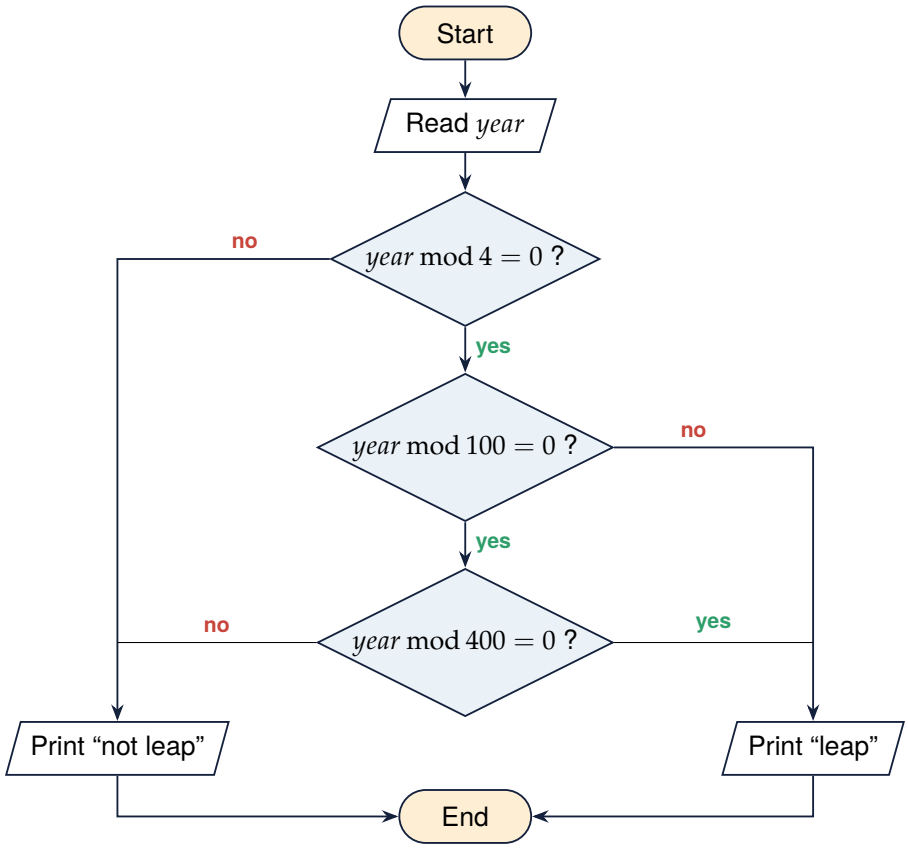
// Method 1: nested decisions (the flowchart on the slide)
if (a > b)
{
    if (a > c)
    {
        Console.WriteLine($"Method 1 (nested): {a}");
    }
    else
    {
        Console.WriteLine($"Method 1 (nested): {c}");
    }
}
else
{
    if (b > c)
    {
        Console.WriteLine($"Method 1 (nested): {b}");
    }
    else
    {
        Console.WriteLine($"Method 1 (nested): {c}");
    }
```

```
}

// Method 2: keep the largest value seen so far
int max = a;
if (b > max)
{
    max = b;
}
if (c > max)
{
    max = c;
}
Console.WriteLine($"Method 2 (running max): {max}");
```

3.6 Example 6 – leap year (three nested decisions)

Gregorian rule: a year is a leap year if it is divisible by 4, except century years, which must be divisible by 400. The chart checks 4, then 100, then 400. The four outcomes join into only two output boxes, placed left (not leap) and right (leap) so that no lines cross.



The same rule as one condition: **IF** ($year \bmod 4 = 0$ **AND** $year \bmod 100 \neq 0$) **OR** $year \bmod 400 = 0$ **THEN** leap.

year	mod4	mod100	mod400	path	leap?
2024	0	24	24	yes, no	yes
2026	2	26	26	no	no
1900	0	0	300	yes, yes, no	no
2000	0	0	0	yes, yes, yes	yes

Listing 6: code/02_06_LeapYear/Program.cs

```
// MCMP113 · Week 2 · Example 6 — Leap year
// Three nested decisions (the flowchart), checked against the equivalent single condition.

Console.Write("Year: ");
int year = int.Parse(Console.ReadLine());

if (year % 4 == 0)
{
    if (year % 100 == 0)
    {
        if (year % 400 == 0)
        {
            Console.WriteLine($"{year} is a leap year");
        }
        else
        {
            Console.WriteLine($"{year} is not a leap year");
        }
    }
    else
    {
        Console.WriteLine($"{year} is a leap year");
    }
}
else
{
    Console.WriteLine($"{year} is not a leap year");
}

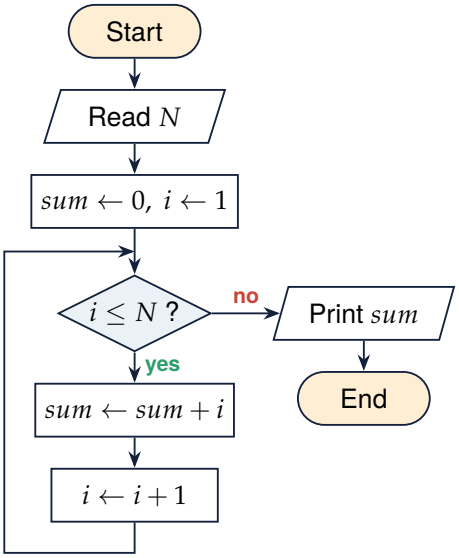
// The same rule as ONE condition (&& = AND, || = OR, != = "not equal")
bool leap = (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
Console.WriteLine($"Single-condition check: {leap}");
```

Four runs

```
Year: 2024
2024 is a leap year
Single-condition check: True
Year: 2026
2026 is not a leap year
Single-condition check: False
Year: 1900
1900 is not a leap year
Single-condition check: False
Year: 2000
2000 is a leap year
Single-condition check: True
```

3.7 Example 7 – the sum $1 + 2 + \cdots + N$ (repetition)

Week 1's Algorithm 3 as a flowchart. The initialisation $sum \leftarrow 0$, $i \leftarrow 1$ happens *once*, before the point where the back-edge enters. A classic mistake is to draw the back-edge above the initialisation box: then sum and i are reset in every pass and the loop never ends.



Three runs

```
N = 100
Sum 1..100 = 5050
N = 5

Sum 1..5 = 15
N = 0
Sum 1..0 = 0
```

The results agree with Gauss's formula $N(N+1)/2$: $100 \cdot 101/2 = 5050$ and $5 \cdot 6/2 = 15$. For $N = 0$ the body never runs (pre-test loop), so the sum stays 0. The full trace for $N = 5$ is in the Week 1 notes.

Listing 7: `code/02_07_SumToN/Program.cs`

```
// MCMP113 · Week 2 · Example 7 — Sum 1 + 2 + ... + N
// Repetition: the arrow from "i <- i + 1" back to the decision becomes a while loop.

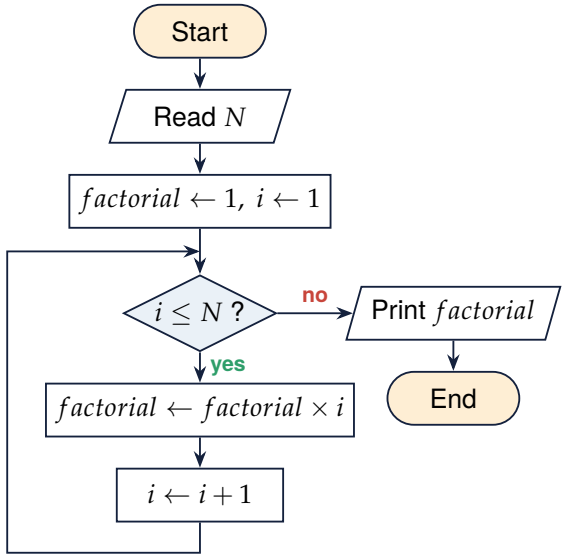
Console.Write("N = ");
int n = int.Parse(Console.ReadLine());

int sum = 0;
int i = 1;
while (i <= n)       // decision box: "yes" enters the loop body
{
    sum = sum + i;
    i = i + 1;       // update: without it the loop never ends
}

Console.WriteLine($"Sum 1..{n} = {sum}");
```

3.8 Example 8 – factorial $N!$

$N! = 1 \times 2 \times \cdots \times N$, with $0! = 1$ by definition. The chart has exactly the shape of the sum; only two things change: the accumulator starts at 1 (the neutral element of multiplication – starting at 0 would always give 0) and the body multiplies.



Three runs

```
N = 5
5! = 120
N = 20
20! = 2432902008176640000
N = 0
0! = 1
```

Listing 8: `code/02_08_Factorial/Program.cs`

```
// MCMP113 · Week 2 · Example 8 — Factorial N! = 1 x 2 x ... x N
// Same loop shape as the sum, but the accumulator starts at 1 and multiplies (long holds up to 20!)
    .

Console.Write("N = ");
int n = int.Parse(Console.ReadLine());

long factorial = 1;
int i = 1;
while (i <= n)
{
    factorial = factorial * i;
    i = i + 1;
}

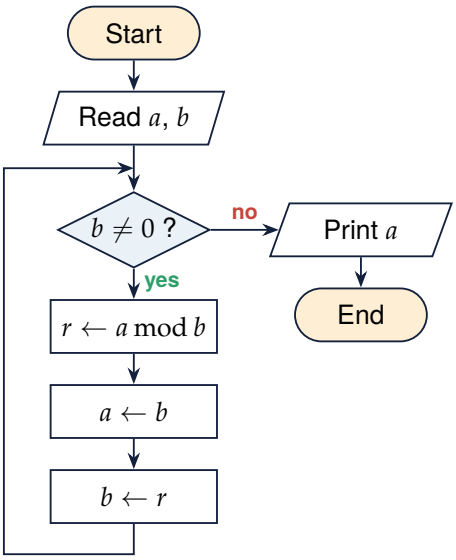
Console.WriteLine($"{n}! = {factorial}");
```

Why `long`?

Factorials grow very fast. The C# type `int` holds values up to $2\,147\,483\,647$, so $12! = 479\,001\,600$ fits but $13! = 6\,227\,020\,800$ does not. The program therefore uses `long` (up to about 9.22×10^{18}), which holds $20! = 2\,432\,902\,008\,176\,640\,000$. $21! \approx 5.1 \times 10^{19}$ overflows even `long` – more on this in Week 6.

3.9 Example 9 – Euclid's greatest common divisor

Week 1's Algorithm 5. This loop is not controlled by a counter: it stops because b strictly decreases ($0 \le a \bmod b < b$) and must reach 0. The order of the three body boxes matters: if $b \leftarrow r$ came before $a \leftarrow b$, both variables would end up equal to r.



a	b	$b \neq 0$?	$r = a \bmod b$
48	18	yes	12
18	12	yes	6
12	6	yes	0
6	0	no	– (print 6)

Sample run

```
a = 48
b = 18
GCD = 6
```

Listing 9: `code/02_09_EuclidGcd/Program.cs`

```
// MCMP113 · Week 2 · Example 9 — Euclid's greatest common divisor (Week 1 algorithm)
// Loop whose condition (b != 0) is not a counter: it ends because b gets smaller in every pass.

Console.Write("a = ");
int a = int.Parse(Console.ReadLine());
Console.Write("b = ");
int b = int.Parse(Console.ReadLine());

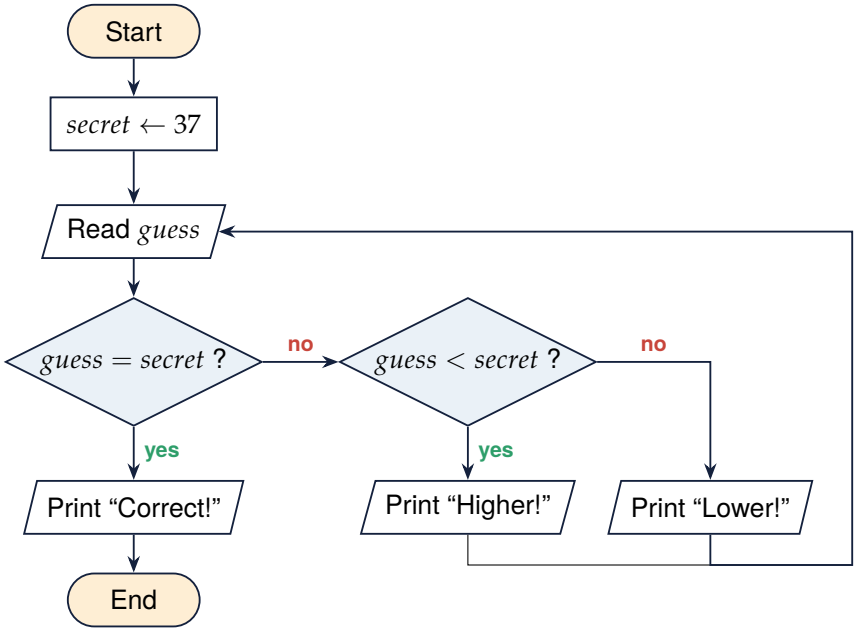
while (b != 0)
{
    int r = a % b;   // r <- a mod b
    a = b;
    b = r;
}

Console.WriteLine($"GCD = {a}");
```

If $a < b$, the first pass swaps them automatically: for $(18, 48)$ the first remainder is $18 \bmod 48 = 18$, giving $a = 48$, $b = 18$, and the program also prints 6.

3.10 Example 10 – guess the number (a decision inside a loop)

The computer holds a secret number; the user guesses until correct and gets the hint "Higher!" or "Lower!" after every wrong guess. Here a two-way decision (the hint) is *nested inside* the loop controlled by "$guess = secret$?". Both hint boxes return to the same input box. In our version the secret is fixed to 37 so that every run is reproducible; in Week 7 it becomes a random number.



Listing 10: `code/02_10_GuessNumber/Program.cs`

```
// MCMP113 · Week 2 · Example 10 — Guess the number
// A loop with a decision inside it: repeat "read, compare, give a hint" until the guess is right.

int secret = 37;   // fixed for now (random number in Week 7)

Console.Write("Your guess: ");
int guess = int.Parse(Console.ReadLine());

while (guess != secret)
{
    if (guess < secret)
    {
        Console.WriteLine("Higher!");
    }
    else
    {
        Console.WriteLine("Lower!");
    }
    Console.Write("Your guess: ");
    guess = int.Parse(Console.ReadLine());   // read again
}

Console.WriteLine("Correct!");
```

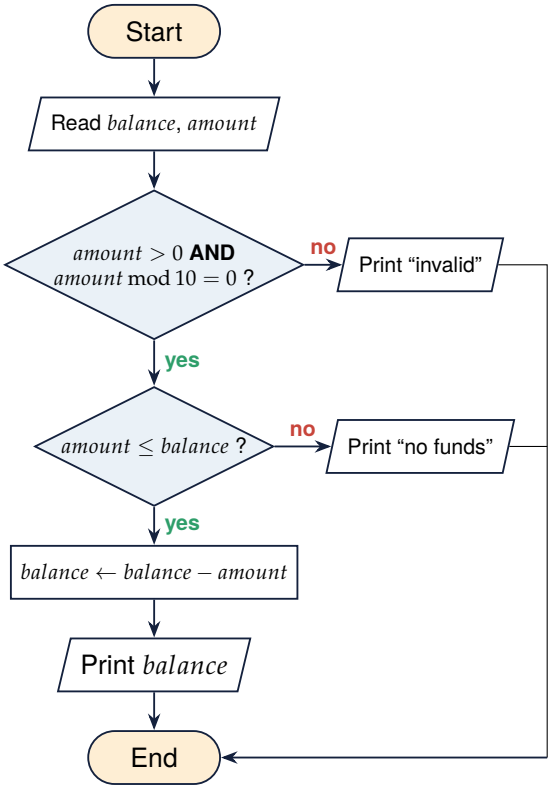
Sample run

```
Your guess: 50
Lower!
Your guess: 25
Higher!
Your guess: 37
Correct!
```

Two details of the translation are worth noticing. (1) The input box is entered from two places (from above and from the back-edge), so in this C# version `ReadLine` appears twice: once before `while` and once at the end of the loop body. (2) The chart leaves the loop on "yes" to *guess* = *secret*; the C# loop *continues* while `guess != secret`. "Exit when X" in a chart becomes "while **NOT** X" in code.

3.11 Example 11 – ATM withdrawal

Week 1's exercise C. The first decision validates the amount (positive and a multiple of 10), the second checks the balance. The two error messages join the main path at End. The C# version follows in Week 5.



Algorithm 3 – ATM withdrawal

```
1  START
2  READ balance, amount
3  IF amount > 0 AND amount mod
   10 = 0 THEN
4      IF amount ≤ balance THEN
5          balance ← balance − amount
6          PRINT balance
7      ELSE
8          PRINT "no funds"
9      END IF
10 ELSE
11     PRINT "invalid"
12 END IF
13 END
```

balance, *amount*	expected output	what it tests
500, 200	300	normal withdrawal
500, 500	0	boundary: the whole balance
500, 510	no funds	not enough money
500, 25	invalid	not a multiple of 10
500, −10	invalid	negative amount

>_ 4 Common mistakes and self-checks

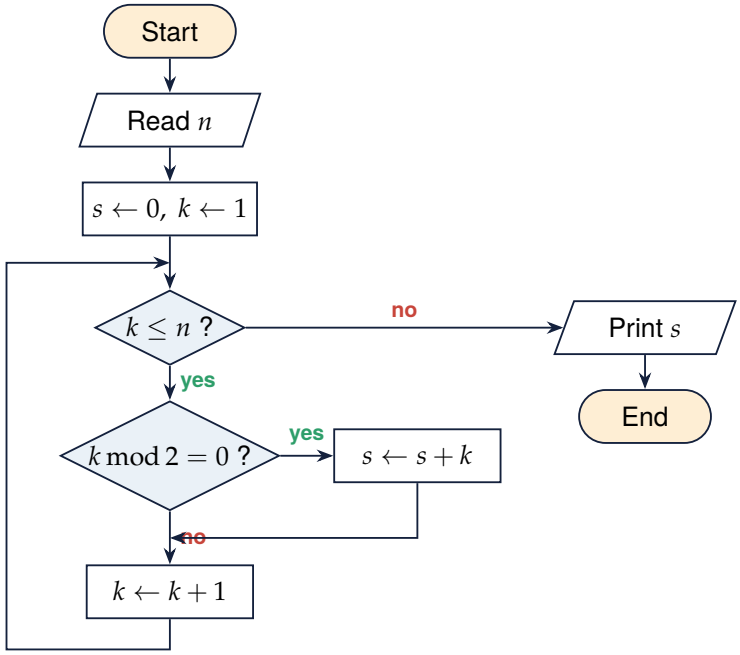
Four flowchart bugs

1. **A decision with one exit.** "$x > 0$?" with only a "yes" arrow: what happens when $x \leq 0$? Every diamond needs two labelled exits.
2. **Missing End / dead end.** A path that stops in the middle of the page. Every path must reach the (single) End terminal.
3. **Loop without update.** "$i \leq N$?" $\rightarrow$ "$sum \leftarrow sum + i$" $\rightarrow$ back, but nothing changes i: the condition stays true forever. Add $i \leftarrow i + 1$. In C# such a program hangs and must be stopped with `Ctrl+C`.
4. **Ambiguous text.** "get the numbers", "calculate it". Write exactly what happens: "Read a, b", "$sum \leftarrow a + b$" (Week 1: *definiteness*).

Other frequent errors: missing arrowheads; exits without yes/no labels; a READ and a calculation in one box; the back-edge of a loop drawn above the initialisation box; branches of an IF that never join again.

4.1 Predict the output

What does the following flowchart print for $n = 6$? (A) 21 (B) 12 (C) 9 (D) nothing, it never stops.



>_ 5 Tools for drawing flowcharts

- **Pen and paper.** Exams are written on paper. Use a pencil and eraser, draw the main path first, and leave generous space between boxes for the branches.
- **draw.io / diagrams.net.** Free, runs in the browser (`https://app.diagrams.net/`) or as a desktop app; it has a *Flowchart* shape library and exports PNG and PDF.
- **Flowgorithm.** Free (Windows). You draw the flowchart and then *run* it step by step while a watch window shows the variables; its source-code viewer shows the same chart in many programming languages, including C#. Flowgorithm adds its own "Declare" shape, because every variable must be declared with a type – a preview of Week 3.
- **Mermaid.** A text-to-diagram language: you write the chart as text and get the picture. GitHub and GitLab render it inside Markdown files; `https://mermaid.live` shows a live preview. The absolute-value chart of Example 2:

```
flowchart TD
  A([Start]) --> B[/Read x/]
  B --> C{"x < 0 ?"}
  C -- yes --> D["x ← −x"]
  C -- no --> E[/Print x/]
  D --> E
  E --> F([End])
```

>_ 6 Summary

1. A **flowchart** is a picture of an algorithm: shapes are kinds of steps, arrows give the order.
2. Six basic symbols (ISO 5807): terminal, input/output, process, decision, predefined process, connector; plus flow lines and annotations.
3. Rules: one Start and one End; two labelled exits per decision; arrows on every line; no crossings; precise text; loops must make progress.
4. Sequence, selection and repetition each have a pattern with one entry and one exit, so patterns can be nested. An upward arrow (back-edge) means a loop.
5. Converting: one box ≈ one pseudocode line; a diamond whose paths join again is an IF; a diamond with a back-edge is a WHILE.
6. Trace tables test a flowchart before any code exists; choose tests that visit every path and every boundary.

Glossary (English – Turkish)

flowchart	akış şeması	flow line	akış çizgisi
symbol	sembol, simge	arrow / arrowhead	ok / ok ucu
terminal (start/end)	başla/bitir simgesi	connector	bağlantı (bağlayıcı) simgesi
input / output	girdi / çıktı	off-page connector	sayfa dışı bağlantı
process	işlem	annotation	açıklama (not)
decision	karar	predefined process	önceden tanımlı işlem (alt program)
sequence	sıralı yapı	nested	iç içe
selection	seçim (karar) yapısı	back-edge	geri dönüş oku
repetition / loop	tekrar / döngü	infinite loop	sonsuz döngü
pre-test loop	ön koşullu döngü	post-test loop	son koşullu döngü
trace table	izleme tablosu	boundary value	sınır değer
counter	sayaç	accumulator	biriktirici (toplam) değişken
leap year	artık yıl	factorial	faktöriyel
remainder (mod)	kalan (mod)	greatest common divisor	en büyük ortak bölen (EBOB)

>_ 7 Exercises

For every exercise: draw the flowchart following the eight rules of Section 1.5, write the matching pseudocode, and trace it with at least one normal and one edge-case input.

7.1 In-class exercises (pairs, 30 minutes)

A. ⋆ **Celsius → Fahrenheit.** Read C; print $F = C \times 9/5 + 32$.

B. ⋆ **Larger of two.** Read a and b; print the larger one, or "equal" if they are the same.

C. ⋆⋆ **Ticket price.** Read an age. Children under 12 pay 50 TL, people aged 65 or over pay 60 TL, everybody else pays 100 TL.

D. ⋆ **Multiplication table.** Read n; print $n \times 1, n \times 2, \ldots, n \times 10$, one per line.

E. ⋆⋆ **Average of N numbers.** Read N, then N numbers one by one; print their average.

F. ⋆⋆ **Guess the number with a counter.** Extend Example 10 so that it also prints how many guesses were needed.

7.2 Homework (due before Week 3)

H1. ⋆ Draw flowcharts for Week 1 exercises A (even/odd), B (leap year – use your own layout; do not copy Example 6) and F (Celsius → Fahrenheit).

H2. ⋆ Trace the GCD flowchart (Example 9) for $(84, 36)$ and for $(17, 5)$ in trace tables.

H3. ⋆⋆ Draw the flowchart of in-class exercise F (guess the number with a counter).

H4. ⋆ Open the `code/` folder and run `02_07_SumToN` and `02_10_GuessNumber` with `dotnet run`. For each run, follow the path through the flowchart with your finger.

7.3 Extra practice

P1. ⋆ **Shopping total with VAT.** Read a unit price and a quantity; print the total including 20 % VAT (KDV).

P2. ⋆ **Find the errors.** A student's chart is: Start → "Read x and double it" → diamond "$x > 10$?" with a single arrow labelled yes to "Print *big*". List every rule that is broken.

P3. ⋆⋆ **Pseudocode → flowchart.** Draw the flowchart of the algorithm below and trace it for $n = 4071$. What does it compute? What does it print for $n = 0$, and is that what we want?

```
1 START
2 READ n
3 count ← 0
4 WHILE n > 0
5     n ← ⌊n/10⌋
6     count ← count + 1
7 END WHILE
8 PRINT count
9 END
```

P4. ⋆⋆ Trace the mystery chart of Section 2.3 for $n = 1000$ and for $n = 1$.

P5. ⋆⋆ For the "predict the output" chart of Section 4, what is printed for $n = 10$? How many times is the condition $k \le n$ evaluated?

P6. ⋆⋆ **Minimum of three.** Draw the running-minimum version of Example 5 (Method 2).

P7. ⋆⋆⋆ **Largest of N numbers.** Read $N \ge 1$ and then N numbers; print the largest. Combine a counting loop with the running maximum.

P8. ⋆⋆⋆ **Power.** Read a number b and an integer $e \ge 0$; compute b^e by repeated multiplication (no power operator). Trace 2^{10} and 5^0.

References and further reading

[1] ISO 5807:1985, *Information processing – Documentation symbols and conventions for data, program and system flowcharts, program network charts and system resources charts.* https://www.iso.org/standard/11955.html

[2] F. B. Gilbreth and L. M. Gilbreth, "Process Charts", *Transactions of the ASME* 43, 1921.

[3] H. H. Goldstine and J. von Neumann, *Planning and Coding of Problems for an Electronic Computing Instrument*, Institute for Advanced Study, 1947.

[4] C. Böhm and G. Jacopini, "Flow diagrams, Turing machines and languages with only two formation rules", *Communications of the ACM* 9(5), 366–371, 1966. https://doi.org/10.1145/355592.365646

[5] Flowgorithm – free beginner's programming language based on flowcharts. http://www.flowgorithm.org/

[6] diagrams.net (draw.io) – online diagram editor. https://app.diagrams.net/

[7] Mermaid – flowchart syntax. https://mermaid.js.org/syntax/flowchart.html

[8] Microsoft, *C# selection statements: if, if-else and switch*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/selection-statements

[9] Microsoft, *C# iteration statements: for, foreach, do and while*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/iteration-statements

[10] Microsoft, *Top-level statements – programs without Main methods*. https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/program-structure/top-level-statements

[11] Microsoft, *Arithmetic operators – remainder operator %*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/arithmetic-operators